CSTE4103 — Cloud Computing

Virtual Machines and Containers

Virtualization, Hypervisors, VM Migration, Docker, and Container Security

Lecture 3 • Cybersecurity Engineering — 4th Year

Dr. Ruaa Ali Khamees • ICT Department
Polytechnic College for Engineering Specializations – Baghdad, Middle Technical University

LECTURE 3 • OVERVIEW

Learning Objectives

1. **Why virtualize**
   Explain consolidation, isolation, and elasticity.
2. **Hypervisors**
   Compare Type 1 and Type 2, and full, para, and hardware-assisted virtualization.
3. **VM lifecycle**
   Describe snapshots, cloning, and live migration.
4. **Containers**
   Explain namespaces and cgroups, and how containers differ from VMs.
5. **Docker**
   Name its components and trace the build, ship, run workflow.
6. **Security**
   Compare isolation strength and attack surface of VMs and containers.

1 — WHY VIRTUALIZE

One Server, Many Machines

Last week's racks hold physical servers. Virtualization slices each one into isolated virtual machines, which is what makes cloud sharing possible.

Consolidation

Typical physical servers idle at low utilization. Packing many VMs on one host raises utilization and cuts hardware, power, and space.

Isolation

Each VM has its own OS and resources. A crash or compromise in one should not reach its neighbors.

Elasticity

A VM is software: create, clone, resize, or move it in minutes. This is the engine behind on-demand IaaS.

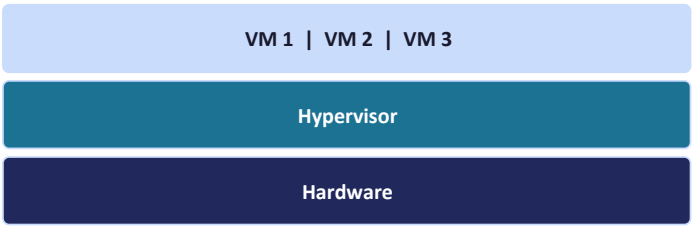
Link to Lab 3: you will see this on your own PC with VMware Workstation.

2 — HYPERVISORS

Type 1 and Type 2 Hypervisors

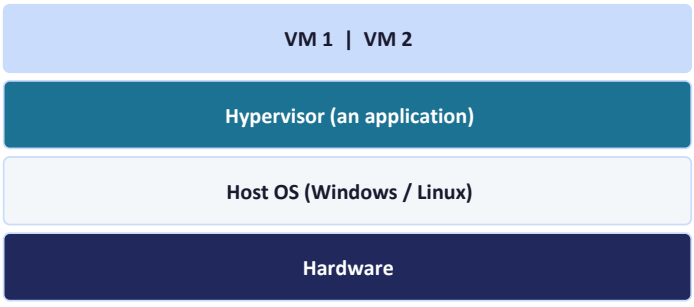
A hypervisor (virtual machine monitor) creates VMs and shares CPU, memory, and I/O among them.

Type 1 — bare metal



Examples: ESXi, Hyper-V, KVM, Xen — used in data centers.

Type 2 — hosted



Examples: VMware Workstation, VirtualBox — used on desktops and labs.

Cloud providers run Type 1 (KVM-based or custom).

Approaches to Virtualization

Approach	How it works	Guest OS changed?	Trade-off
Full virtualization	Hypervisor emulates the full hardware; binary translation for privileged instructions.	No	Compatible, but slower
Para-virtualization	Guest knows it is virtual and calls the hypervisor directly (hypercalls).	Yes	Fast, needs modified OS
Hardware-assisted	CPU extensions (Intel VT-x, AMD-V) trap privileged operations in hardware.	No	Fast and compatible; today's default

⚠ Check: If Intel VT-x / AMD-V is disabled in BIOS, 64-bit guests may fail to start in your lab. Enable it first.

What the Hypervisor Virtualizes

CPU

Schedules vCPUs onto physical cores. Over-subscription lets total vCPUs exceed cores, which is efficient but can cause contention.

Memory

Maps guest memory to host memory. Techniques such as ballooning and page sharing reclaim unused RAM.

I/O

Presents virtual disks and virtual NICs. Disks are often files on shared storage; NICs connect to virtual switches.

Security consequence: whoever controls the hypervisor controls every VM above it. It is the most valuable target on the host, so keep it minimal and patched.

VM Lifecycle: Snapshots and Clones

A VM is mostly files: virtual disks plus a configuration. That is why VMs are easy to copy and recover.

Create

Pick vCPU, RAM, disk, network; install or use an image.

Run

Start, pause, suspend, resize resources.

Snapshot

Save the state at a point in time to roll back after a mistake or failed update.

Clone

Copy a VM or a golden template to deploy many identical machines.

Retire

Delete the VM and securely wipe its disks.

⚠ Security: Snapshots and clones keep old data and old secrets (keys, passwords, unpatched software). Treat them like backups and protect them the same way.

VM Migration

Moving a VM between hosts lets the provider balance load, patch hosts, and evacuate failing hardware.

Cold migration

Shut the VM down, copy it, start it on the new host. Simple, but there is downtime.

Live migration

Move a running VM with almost no downtime; users usually do not notice.

Live migration, pre-copy method

1. Copy all memory pages while the VM keeps running
2. Re-copy pages that changed (repeat)
3. Pause briefly, send the last pages and CPU state
4. Resume on the new host

Memory is sent over the network: encrypt the migration channel.

4 — CONTAINERS

Containers: Isolation Without a Guest OS

A container is an ordinary process on the host kernel, wrapped by two Linux features that limit what it can see and use.

Namespaces — what it can SEE

Separate views of processes (PID), network, mounts, hostname, and users. Inside, the container thinks it is alone.

cgroups — what it can USE

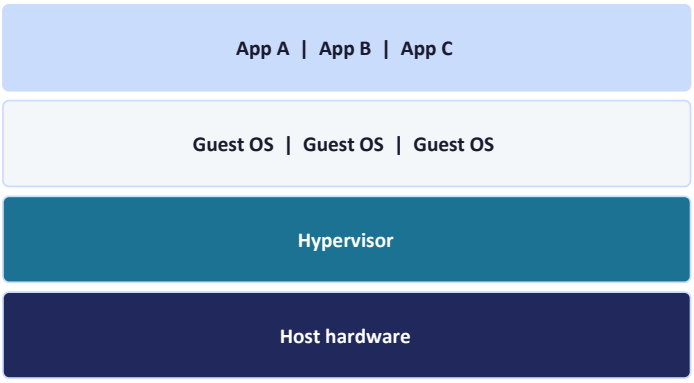
Limits CPU, memory, and I/O per container so one cannot starve the others.

Because there is no guest OS, a container starts in seconds, uses megabytes, and you can pack many more per host than VMs. The price is a shared kernel.

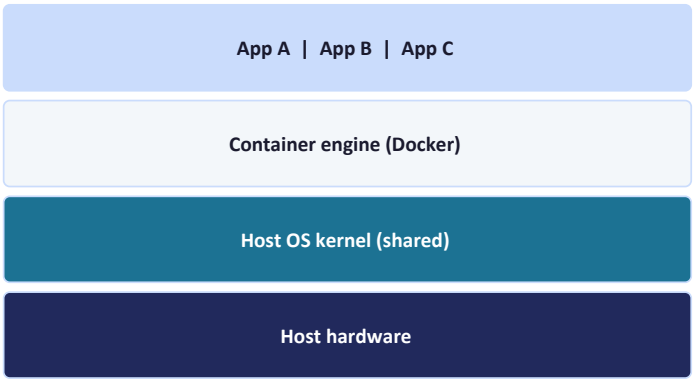
4 — CONTAINERS

VM Stack vs Container Stack

Virtual machines



Containers



Each VM carries a full OS (GBs, minutes to boot). Containers share one kernel (MBs, seconds to start).

4 — CONTAINERS

VMs vs Containers

Aspect	Virtual machine	Container
Isolation	Strong: separate kernel per VM	Weaker: shared host kernel
Startup	Minutes	Seconds
Size	Gigabytes	Megabytes
Density per host	Tens	Hundreds
Guest OS	Any (Windows, Linux...)	Same kernel family as host
Best for	Legacy apps, strong tenant separation	Microservices, CI/CD, scale-out

In practice they combine: containers usually run inside VMs in the cloud.

5 — DOCKER

Docker Components

Docker packages an application and everything it needs, so it runs the same on a laptop, a lab server, or a cloud VM.

Dockerfile

A text recipe: base image, files, commands.

Image

Read-only layered template built from the Dockerfile.

Container

A running instance of an image, with a thin writable layer.

Engine (daemon)

Builds images and runs containers on the host.

Registry

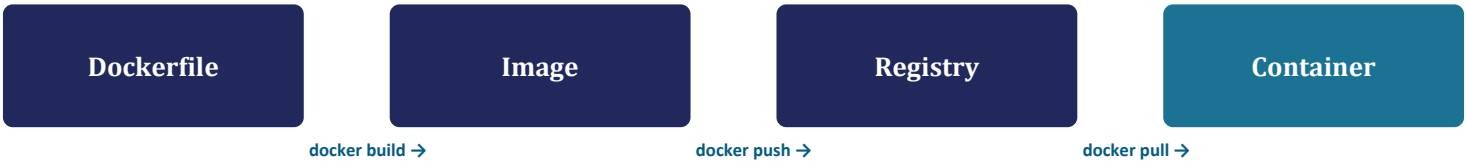
Stores and distributes images (Docker Hub, private registry).

Volume / Network

Persistent data outside the container; virtual networks between containers.

5 — DOCKER

Build, Ship, Run




```
# Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/

# build and run
docker build -t mysite .
docker run -d -p 8080:80 mysite
```

What happened: the image holds the web server and your page; the container runs it and maps host port 8080 to container port 80.

6 — SECURITY LENS

Isolation and Attack Surface

Virtual machines

Risks: VM escape through a hypervisor flaw, side-channel attacks between co-located VMs, stale snapshots, VM sprawl.

Containers

Risks: kernel exploit reaches every container, privileged containers, vulnerable or untrusted images, secrets baked into images.

Good practices

- Patch the hypervisor and host kernel; keep them minimal.
- Use trusted, scanned images; pin versions; never store secrets in an image.
- Do not run containers as root or with --privileged; drop capabilities.
- Separate sensitive workloads on different hosts or inside VMs.

7 — APPLY IT

VM or Container? Mini Case

The scenario

A university hosts three things: (1) an old Windows-only grading system, (2) a student web portal with traffic spikes at registration, (3) a lab where outside researchers run untrusted code.

Choose VM or container for each, and justify it.

1. **Needs another OS or old kernel?**
   Points to a VM.
2. **Needs fast scale-out?**
   Points to containers.
3. **Runs untrusted code?**
   Strong isolation: a VM.
4. **Can we combine?**
   Containers inside VMs is common.

SUMMARY

Summary and Next Week

Key points

- Virtualization enables consolidation, isolation, and elasticity.
- Type 1 hypervisors run on hardware; Type 2 run on a host OS. Hardware-assist is the norm.
- VMs are files: snapshot, clone, and live-migrate them, and protect those copies.
- Containers share the host kernel via namespaces and cgroups: light and fast, weaker isolation.
- Docker: Dockerfile builds an image, a registry stores it, an engine runs it as a container.

Next week (Lecture 4)

Virtual Networks and Virtual Storage: virtual switches, overlays and VLANs, SDN, block, file, and object storage.

Reference: Comer, D. E. (2021). The Cloud Computing Book. CRC Press.